

BAPTISM: A Robust Framework for Encrypted Malicious Traffic Identification With Low-Quality Training Data

Xiang Luo¹, Chang Liu¹, Gang Xiong¹, Gaopeng Gou, Zhen Li, Junzheng Shi, Li Guo, and Binxing Fang²

Abstract—Machine learning (ML) is highly effective for accurate encrypted malicious traffic identification by using high-quality training data. In fact, obtaining such data is costly and challenging. As a result, many ML-based models are inevitably trained on low-quality data and perform poorly. To enhance performance, some methods utilize various sample selection techniques to choose confident samples for model training. However, they often rely on a single metric for this selection, which restricts their adaptability across diverse datasets and noise conditions. In this paper, we propose a robust framework BAPTISM for identifying encrypted malicious traffic with low-quality training data. Particularly, BAPTISM selects a suitable base model for each task, and trains it with early stopping to generate traffic representation before overfitting occurs. Then, we devise an adaptive metric selection strategy to select confident samples. By employing two metrics (JSD and CSD) to assess the characteristic of traffic representation from distinct perspectives, we find the more proper metric for each class and apply it for confident sample selection. According to the confident samples and selected metric for each class, we develop a label correction tactic which adapts to class nature to improve the quality of training data. Finally, we employ parallel training strategy to train the base model with the corrected data, further mitigating the impact of low-quality data. We conduct experiments across three real-world malicious traffic datasets with various noise settings. The results demonstrate that BAPTISM is compatible with different base models and outperforms across noise ratios ranging from 20% to 90%. Meanwhile, BAPTISM consistently selects the confident samples with the highest purity and volume under each setting.

Index Terms—Encrypted malicious traffic, noisy label, low quality data, intrusion detection, deep learning.

I. INTRODUCTION

MALICIOUS network traffic has been implicated in numerous security incidents [1], such as data breaches and extortion, underscoring the critical need for effective identification methods. Moreover, Zscaler indicates that

approximately 87% of cyber threats are now delivered over encrypted channels in 2024 [2], seriously challenging the conventional methods based on plaintext payload analysis [3]. To address this issue, contemporary research has pivoted toward machine learning (ML) techniques that leverage behavioral characteristics and side-channel information to develop effective identification models by supervised-learning [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15].

However, all the mentioned ML-based methods rely on high-quality training data. For clarity, in this paper, datasets with fully accurate labels are termed high-quality data, and samples with correct labels are termed clean samples. Conversely, datasets containing mislabeled samples (also referred to as noisy samples) are considered low-quality data. In practice, ensuring label correctness in malicious traffic datasets presents significant challenges. Active annotation methods, which rely on expert knowledge, not only incur high manual costs but also heavily depend on the experts' personal interpretation of the data. Moreover, attackers continuously employ techniques such as encryption and obfuscation to conceal network traffic characteristics, further complicating accurate dataset annotation. On the other hand, passive annotation methods, which leverage host-based forensic logs and external threat intelligence feeds (e.g., IP blacklists), become less reliable over time due to the constantly evolving nature of cyber-attacks. Therefore, regardless of the annotation method employed, noisy samples inevitably arise. As a result, ML-based methods are inevitably trained on low-quality data, leading to performance degradation.

To tackle the issue of low-quality training data, various approaches have been proposed. The core idea behind these methods is to first select a subset of samples with highly reliable labels (referred to as confident samples) based on their designed separation metric, or subsequently leverage these samples to improve the overall label quality of the dataset before model training. Therefore, these methods can be divided into confidence-based methods [16], [17], [18], [19] and feature-based methods [20], [21], [22], [23], [24], [25] based on their separation metrics. Confidence-based methods focus on the local patterns, whereas feature-based methods consider the global patterns of latent features. Nevertheless, existing methods still face several common issues: (1) **These methods are dependent on specific model architectures** and cannot be easily adjusted for different identification tasks. (2) **These methods typically rely solely on a fixed metric for**

Received 23 March 2025; revised 19 September 2025; accepted 16 December 2025. Date of publication 24 December 2025; date of current version 14 January 2026. This work was supported by the National Natural Science Foundation of China under Grant 62402492. The associate editor coordinating the review of this article and approving it for publication was Dr. Li Lu. (Corresponding author: Chang Liu.)

Xiang Luo, Chang Liu, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Li Guo are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: luoxiang@iie.ac.cn; liuchang@iie.ac.cn; xionggang@iie.ac.cn; gougaopeng@iie.ac.cn; lizhen@iie.ac.cn; shijunzheng@iie.ac.cn; guoli@iie.ac.cn).

Binxing Fang is with Hainan Province Fang Binxing Academician Workstation, Hainan 571924, China (e-mail: fangbx@cae.cn).

Digital Object Identifier 10.1109/TIFS.2025.3648170

confident sample selection without evaluating its suitability. (3) **These methods results in a limited number of confident samples** being available for model training, which adversely affects the efficiency.

In this paper, we introduce a novel framework BAPTISM for encrypted malicious traffic identification with low-quality training data. BAPTISM operates independently of specific model architectures, allowing for choosing an appropriate base model for each task. It consists of four coupled components. First, we implement early representation by using the base model trained with early stopping, generating traffic representation before overfitting on noisy samples. Second, we propose an adaptive metric selection strategy for confident sample selection in each class instead of using single metric. In particular, we utilize both Jensen-Shannon Divergence (JSD) and Confidence Sample Distance (CSD) to evaluate traffic representation nature jointly. We assess the separation performance of these two metrics by analyzing the mean and variance of JSD values between sample clusters for each class, then select the more suitable one and apply for confident sample selection. Third, we conduct label correction tailored to the confident samples and the selected metric for each class, thereby improving the label quality of the training data. In addition to utilizing a centroid-based correction approach, we develop a KNN-based correction method specifically designed for traffic classes where JSD is used for selecting confident samples, better accommodating their loose distribution in latent space. Fourth, we apply parallel training to train a the base model with the corrected data, further reducing the impact of noisy samples. We initialize two base models with different parameters and train them simultaneously. During each training epoch, we dynamically select low-loss samples and mutually update each other's parameters. We finally select the better-performing one for identification.

In summary, our contributions are as follows:

- We propose a novel framework BAPTISM for identifying encrypted malicious traffic using low-quality training datasets. BAPTISM is independent of specific model architectures and adapts to different base models for each task. Moreover, it integrates early stopping and parallel training strategies to mitigate the impact of noisy samples.
- We propose an adaptive metric selection strategy for confident sample selection instead of using only one metric. We assess the separation performance of JSD and the CSD by analyzing the characteristics of the divided clusters, which enables us to select the more proper metric for each class and effectively filter confident samples.
- We introduce a label correction tactic tailored to the selected metrics and confident samples to improve dataset quality. We establish varying label correction strategies based on the reliability of the centroids of the confident samples, including directly using the nearest centroid's class or seeking additional nearby confident samples.
- We evaluate BAPTISM against various comparison methods on different datasets with various noise ratios. The results show that BAPTISM adapts to different base models and outperforms across all experimental settings. Additionally, it effectively selects confident

samples with optimal purity and volume across various settings.

The rest of this paper is organized as follows: Section II introduces the related work on encrypted malicious traffic identification. Section III clarifies the problem description. Section V presents our proposed BAPTISM framework. Section VI reports experimental results and analysis. Section IX discusses the false positives. Section X gives the conclusion.

II. RELATED WORK

A. Traditional Encrypted Malicious Traffic Identification

Encrypted malicious traffic hides the plaintext payload, preventing identification via traditional methods like Deep Packet Inspection (DPI) [3]. To solve this problem, a series of methods have been developed, most of them share a similar design that profiles the behavior of encrypted traffic via manual feature engineering or automatic feature extraction and builds a ML-based classifier. For example, Rong et al. [9] propose an ensemble learning-based framework MalFinder for malicious traffic detection. They utilize statistical features and sequence features to describe the traffic while expanding the dimensions of these two types of features to enhance their capability for representing traffic data. Fu et al. [11] propose Whisper that utilizes the sequential information of network traffic represented by the frequency domain features to achieves high detection accuracy.

With the development of deep learning technology, Wang et al. [12] propose a novel intrusion detection system HAST-IDS, which first learns the low-level spatial features of network traffic using deep convolutional neural networks (CNNs) and then learns high-level temporal features using long short-term memory networks (LSTM). Cai et al. [14] propose BiTCN_MHSA, which utilizes the multi-head self-attention mechanism to assign different weights to different subsequence of network traffic. This makes the model more focus on the characteristics of malicious network traffic. Hong et al. [15] propose MalDiscovery which constructs an attribute KNN graph according to the similarity of network traffic features to identify malicious traffic. To improve model stability and generalization across different network environments and time periods, Han et al. [26] propose an encrypted malicious traffic detection approach that combines reconstructive domain adaptation with adversarial hybrid neural networks, achieving effective cross-domain detection in 5G environments.

The effectiveness of these methods relies on the correctness of the labels. Thus, the performance of these methods will be seriously affected by the low-quality training data.

B. Encrypted Malicious Traffic Identification Under Low-Quality Training Data

Inaccurate labels undermines the training of encrypted malicious traffic identification models, resulting in biased distribution learning. Consequently, numerous research efforts are directed towards addressing low-quality data by selecting a subset of confident samples for model training. Based on the way to select confident samples, they can be categorized into two types: confidence-based and feature-based methods.

TABLE I
SUMMARY OF RELATED WORK ON ENCRYPTED MALICIOUS TRAFFIC IDENTIFICATION

Methods	Noise Handling	Sample Selection	Label Correction	Fine-grained Identification	Model Agnostic
MalFinder [9]	○	○	○	○	○
Whisper [10], [11]	○	○	○	○	●
Hast-IDS [12], Cbseq [13]	○	○	○	●	○
BiTCN_MHSA [14]	○	○	○	●	○
MalDiscovery [15]	○	○	○	○	○
UTC [16], NSTC [17]	●	○	○	●	○
MMCo [18], CMC [19]	●	●	○	●	○
ULDC [20]	●	○	○	○	○
BoAu [21], MCRc [22]	●	○	○	●	○
Gedss [23], [24]	●	●	○	●	○
RAPIER [25]	●	●	●	○	○
BAPTISM	●	●	●	●	●

● means totally support, and ○ means not support.

Confidence-based methods address the noisy label problem by analyzing the confidence scores of model predictions to identify confident samples. For example, Wang et al. [16] propose a consensus algorithm combined with a set of classifiers to filter noisy labeled traffic data. Furthermore, they improve the performance of noise elimination by estimating the reliability of the remaining training data before it builds a robust traffic classifier [17]. On the other hand, MMCo [18] and CMC [19] leverage multi modal information in heterogeneous networks to detect malicious traffic with noisy labels. During training, they identify samples on which the two networks make divergent predictions, as these samples are considered more informative. Only these high-information samples are used for updating model parameters, thereby enhancing the model's robustness to noisy samples.

Feature-based methods comprehensively analyze the distribution of latent features to select confident samples. For instance, ULDC [20] uses unsupervised neural networks to map samples to a low-dimensional space and the difference of these low-dimensional embeddings to evaluate the confidence of each sample. Yuan et al. propose BoAu [21], which alleviates the effect of noisy samples by encouraging the projections of hard samples to move closer to the decision center while increasing the inter-cluster distance during training. Building on this idea, they further introduce MCRc [22], which incorporates three different levels of constraints to achieve robust feature representation and enhance resilience to noisy samples. In addition, Yao et al. propose Gedss [23], which employs a JSD-based sample selection strategy combined with a semi-supervised learning approach to improve model performance in the presence of noisy labels. They further develop a novel two-stage framework for malicious traffic detection [24], which introduces a double-constrained similarity rule to provide a more comprehensive measure of similarity between small-loss samples. Additionally, their method constructs a topological graph to refine low-quality training data, further improving detection robustness. Meanwhile, RAPIER [25] is developed to fully utilize the different distribution of normal and malicious traffic data in the feature space to augment new data for model training in the case that the amount of training data is limited and non-negligible noisy samples exist.

We compare the aforementioned methods with BAPTISM across different functional aspects, including Noise Handling (whether the method can address noisy samples), Sample

Selection (whether the method can filter confident samples), Label Correction (whether the method can correct sample labels), Fine-grained Identification (whether the method can perform fine-grained identification of malicious traffic), and Model Agnostic (whether the method is independent of specific model architectures). The summary results are shown in Table I. For methods that do not address noisy samples [9], [10], [11], [12], [13], [14], [15], their performance is significantly impacted by low-quality training data. While other comparative methods [16], [17], [18], [19], [20], [21], [22], [23], [24], [25] do address noisy samples, they fail to comprehensively cover sample selection, label correction, and fine-grained identification. Moreover, they are dependent on specific model architectures, which limits their applicability.

III. PROBLEM DESCRIPTION

In this section, we formalize the problem of identifying malicious traffic using low-quality training data. Consider a malicious traffic dataset $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, \hat{y}_i)\}_{i=1}^{N_{\text{train}}}$ used for training, where $\mathbf{x}_i \in \mathbb{R}^D$ represents the i -th malicious network flow, and $\hat{y}_i \in \{1, \dots, C\}$ is the observed label associated with $\mathbf{x}_i$. For each $\mathbf{x}_i$, there exists a single latent true label, denoted as y_i , which may or may not be consistent with the observed label $\hat{y}_i$. To clarify, we define network flows with correct labels ($y_i = \hat{y}_i$) as clean samples, while those with incorrect labels ($y_i \neq \hat{y}_i$) are referred to as noisy samples.

Our goal is to train a model for the identification of encrypted malicious traffic using $\mathcal{D}_{\text{train}}$ that contain noisy samples. The model will then be evaluated on a test dataset $\mathcal{D}_{\text{test}} = \{\mathbf{x}_i\}_{i=1}^{N_{\text{test}}}$ to assess its ability to accurately detect malicious traffic.

IV. PRELIMINARY ANALYSIS

To motivate our proposed method, we first conduct a preliminary study to examine whether confident-based metric and distribution-based metrics, which are two types of common metric have been used to select confident samples introduced in Section II. Confident-based metrics (such as Jensen-Shannon Divergence (JSD) and Cross Entropy Loss (CE Loss)) capture local differences, while feature-based metric (such as distance to class centroid) reflect global relationships.

We compare two representative baselines: CMC [19], which selects confident samples solely based on CE loss, and MCRc

TABLE II
COMPARISON OF CMC AND MCRE ON SELECTING CONFIDENT SAMPLES UNDER NOISE RATIO OF 40%

Datasets	Traffic Class	Precision		Recall		F1	
		CMC	MCRE	CMC	MCRE	CMC	MCRE
Malicious_TLS	Caphaw.A_None_TLS_CC	1.000	1.000	0.931	0.551	0.964	0.711
	CobaltStrike_None_TLS_CC	1.000	0.999	0.725	0.991	0.841	0.995
	Panda.BZA/tr_None_TLS_CC	1.000	1.000	0.671	0.973	0.803	0.986
	Tor_None_TLS_CC	1.000	1.000	0.793	0.970	0.885	0.985
CICIDS-2017	Bot	0.999	0.927	0.591	0.985	0.743	0.955
	DoS Hulk	1.000	1.000	0.937	0.665	0.967	0.799
	DoS Slowhttptest	1.000	0.971	0.816	0.918	0.898	0.944
	PortScan	1.000	0.976	0.926	0.747	0.962	0.846
DoHBrw	Benign	0.936	0.673	0.846	0.944	0.889	0.786
	dns2tcp	0.990	0.829	0.810	0.880	0.891	0.854
	dnscat2	0.989	0.825	0.513	0.774	0.675	0.799
	iodine	0.990	0.858	0.562	0.772	0.717	0.813

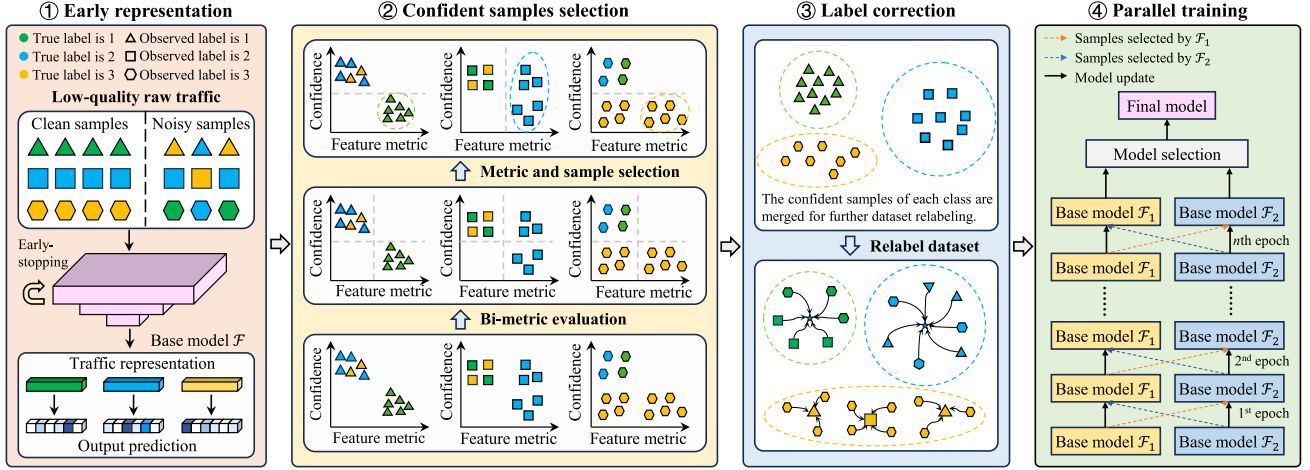


Fig. 1. The overall workflow of BAPTISM. In this figure, Confidence represents the confidence-based metrics, Feature metric represents the feature-based metric, and GMM represents Gaussian Mixture Model.

[22], which relies exclusively on the distance to the class centroid. To evaluate their effectiveness in confident sample selection, we report three metrics: Precision, the proportion of clean samples among the selected ones; Recall, the proportion of clean samples correctly selected within each class; and F1, the harmonic mean of Precision and Recall. Experiments are conducted on the Malicious_TLS, CICIDS-2017, and DoHBrw2020 datasets with a noise ratio of 40%. The results, summarized in Table II, demonstrate the sample selection performance of CMC and MCRE for representative malicious traffic classes across different datasets.

The results reveal that the effectiveness of the two methods varies by class. From the perspective of the overall F1 score, CMC achieves clearly better results on certain classes such as Caphaw.A_None_TLS_CC and DoS Hulk, whereas MCRE performs notably better on classes such as Panda.BZA/tr_None_TLS_CC and Bot. This observation highlights the complementarity of the two metrics. Looking at the individual metrics, CMC generally yields higher Precision, indicating that the samples it selects tend to have higher purity. In contrast, MCRE achieves higher Recall in several cases, suggesting that it is able to retain a larger fraction of clean samples within each class.

In summary, these results demonstrate that confidence-based and feature-based selection strategies emphasize different aspects of the data, providing complementary strengths.

V. METHODOLOGY

In this paper, we propose a robust framework BAPTISM to address the problem of low-quality training data in encrypted malicious traffic identification. It consists of four components as shown in Figure 1. The early representation component uses the appropriate base model trained with early stopping to generate traffic representation. Next, the confident sample selection component evaluates the characteristic of traffic representation based on bi-metrics and selects the more proper one to filter confident samples. Then, the label correction component improves the label quality of training data by applying correction tactic tailored to the confident samples and selected metric of each class. Finally, the parallel training component trains the base model with the corrected data. We introduce each component in detail in the following subsections.

A. Early Representation

Early representation component aims at selecting the appropriate base model $\mathcal{F}$ for each identification task and use it to generate the traffic representation for each sample in $\mathcal{D}_{\text{train}}$ before overfitting occurs with noisy samples. Guided by empirical evidence suggesting that neural networks prioritize learning clean patterns before memorizing noisy samples [27], we implement early stopping during training the base model with cross-entropy (CE) loss to prevent overfitting on noisy

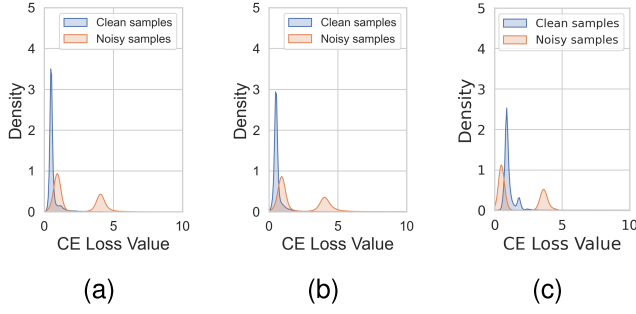


Fig. 2. Cross-Entropy (CE) loss distribution for noisy and clean samples when training the base model for different epochs on Malicious_TLS dataset with a noise ratio of 0.4. (a) Stop at epoch 10; (b) Stop at epoch 50; (c) Stop at epoch 100.

samples. Using the Malicious_TLS dataset as an example, Figure 2 illustrates how early stopping prevents overfitting to noisy samples at a noise ratio of 0.4. At epoch 10, there is a clear difference in CE loss between noisy and clean samples. As training progresses to epoch 50, the model increasingly adapts to the noisy samples, and the density of clean samples with low loss decreases. By epoch 100, some noisy samples have a lower CE loss than clean samples, indicating that the model has learned incorrect patterns.

To this end, we restrict the training to a minimal number of initial epochs, typically ranging from 10 to 20 epochs. This ensures that $\mathcal{F}$ preliminarily extracts traffic representation from each sample with correct patterns.

B. Confident Sample Selection

Based on traffic representation, we select confident samples from each class in this component. This process is conducted in two stages. In the first stage, we evaluate the characteristics of traffic representations from two perspectives: the confidence of the model predictions and the feature distribution of the samples, and divide samples per class into two clusters based on these evaluations. In the second stage, we individually assess the effectiveness of each perspective in separating clean and noisy samples, and select the most suitable one to filter confident samples.

Stage 1: bi-metric evaluation. We utilize JSD and CSD metrics to evaluate the traffic representation as they are complementary [28]: JSD is calculated based on prediction scores, which represent local characteristics, whereas CSD considers the distribution of traffic representation, reflecting global properties. By jointly utilizing both metrics, we are able to address the limitation of relying on only one metric, ensuring a more effective evaluation of traffic representation from distinct perspective.

To compute these two metrics, we first partition the training dataset $\mathcal{D}_{\text{train}}$ into class-specific subsets $\mathcal{D}_c = \{(\mathbf{x}, \hat{\mathbf{y}}) | \hat{\mathbf{y}}_c = c\}$ for each class c , where $c = 1, \dots, C$, based on the observed labels. For each instance $(\mathbf{x}_i, \hat{\mathbf{y}}_i)$ within $\mathcal{D}_c$, we utilize our trained base model $\mathcal{F}$ to obtain both the traffic representation vector $\mathbf{f}_i$ and the corresponding probability distribution $\mathbf{p}_i = [p_i^1, p_i^2, \dots, p_i^C]$. Here, p_i^j denotes the predicted probability that the instance belongs to class j .

JSD produced by base model $\mathcal{F}$ on sample $\mathbf{x}_i$ is defined as:

$$\text{JSD}(\mathbf{x}_i) = \frac{1}{2} \text{KL} \left(\mathbf{p}_i \parallel \frac{\mathbf{p}_i + \hat{\mathbf{y}}_i}{2} \right) + \frac{1}{2} \text{KL} \left(\hat{\mathbf{y}}_i \parallel \frac{\mathbf{p}_i + \hat{\mathbf{y}}_i}{2} \right) \quad (1)$$

where $\text{KL}(\cdot \parallel \cdot)$ denotes the Kullback-Leibler divergence, and $\hat{\mathbf{y}}_i$ represents the one-hot encoded observed label.

CSD quantifies the similarity between a sample and its observed class centroid. For each class c , we establish its confident feature centroid $\mathbf{o}_c$ by averaging feature vectors of confident samples:

$$\mathbf{o}_c = \frac{1}{|\mathcal{D}_c^H|} \sum_{i=1}^{|\mathcal{D}_c^H|} \mathbf{f}_i, \quad \mathcal{D}_c^H = \{\mathbf{x}_i | \mathbf{x}_i \in \mathcal{D}_c, p_i^c > \tau_c\} \quad (2)$$

where τ_c serves as the adaptive threshold computed through:

$$\tau_c = \frac{1}{|\mathcal{D}_c|} \sum_{i=1}^{|\mathcal{D}_c|} w_i \times p_i^c, \quad w_i = \max(1, p_i^c / \bar{p}_c^c) \quad (3)$$

Here, $\bar{p}_c^c = \frac{1}{|\mathcal{D}_c|} \sum_{\mathbf{x} \in \mathcal{D}_c} p_i^c$ represents the mean confidence for class c . The weighting scheme w_i prioritizes samples with above-average confidence to enhance cluster purity. During training, both centroids $\mathbf{o}_c$ and confident sample set $\mathcal{D}_c^H$ are dynamically updated alongside model parameters.

Finally, the CSD metric for $\mathbf{x}_i$ is computed as

$$\text{CSD}(\mathbf{x}_i) = \cos(\mathbf{f}_i, \mathbf{o}_{\hat{\mathbf{y}}_i}) = \frac{\mathbf{f}_i^T \mathbf{o}_{\hat{\mathbf{y}}_i}}{\|\mathbf{f}_i\| \|\mathbf{o}_{\hat{\mathbf{y}}_i}\|} \quad (4)$$

This metric measures the cosine similarity between $\mathbf{f}_i$ and $\mathbf{o}_{\hat{\mathbf{y}}_i}$.

After calculating JSD and CSD for all samples, we employ a Gaussian Mixture Model (GMM) to divide the samples in $\mathcal{D}_c$ into two clusters based on these two metrics, respectively, for each class c in the training data. For example, based on JSD, the samples in $\mathcal{D}_c$ can be divided into two clusters: $\mathcal{C}_{\text{JSD}_c}^1$ and $\mathcal{C}_{\text{JSD}_c}^2$. Similarly, based on CSD, the samples in $\mathcal{D}_c$ can be divided into $\mathcal{C}_{\text{CSD}_c}^1$ and $\mathcal{C}_{\text{CSD}_c}^2$. For both $\mathcal{C}_{\text{JSD}_c}$ and $\mathcal{C}_{\text{CSD}_c}$, we assume that cluster 1 corresponds to the cluster with the larger mean value, while cluster 2 corresponds to the cluster with the smaller mean value.

Stage 2: metric and sample selection. Based on the separation results of JSD and CSD, we evaluate their performance and select the more suitable metric to filter confident samples. We can simplify this problem by determining whether JSD is the proper metric to separate clean samples from noisy samples. To achieve this, we compute the mean and variance of the JSD values within the two clusters $\mathcal{C}_{\text{CSD}_c}^1$ and $\mathcal{C}_{\text{CSD}_c}^2$. Specifically, μ_1 and σ_1 represent the mean and variance of JSD values for samples in $\mathcal{C}_{\text{CSD}_c}^1$, while μ_2 and σ_2 represent those for samples in $\mathcal{C}_{\text{CSD}_c}^2$. Generally, the cluster $\mathcal{C}_{\text{CSD}_c}^1$ consists of samples with high similarity of feature centroids $\mathbf{o}_c$, which contains a higher proportion of clean samples, leading to a lower mean JSD μ_1 . Conversely, $\mathcal{C}_{\text{CSD}_c}^2$ may include more noisy samples, resulting in a higher mean JSD μ_2 . We present different conditions of proper metric selection in Figure 3 and provide a detailed illustration:

(1) Condition 1: JSD is the proper metric. If $\sigma_1 < \sigma_2$ and $\mu_1 < d < \mu_2$ where d is the separation threshold between $\mathcal{C}_{\text{JSD}_c}^1$ and $\mathcal{C}_{\text{JSD}_c}^2$, this indicates the presence of potential clean samples in $\mathcal{C}_{\text{CSD}_c}^2$, which increases the standard deviation of

△ Labeled as 1 □ Labeled as 2 ◇ Labeled as 3 ● Ground truth is 1 ● Ground truth is 2 ● Ground truth is 3

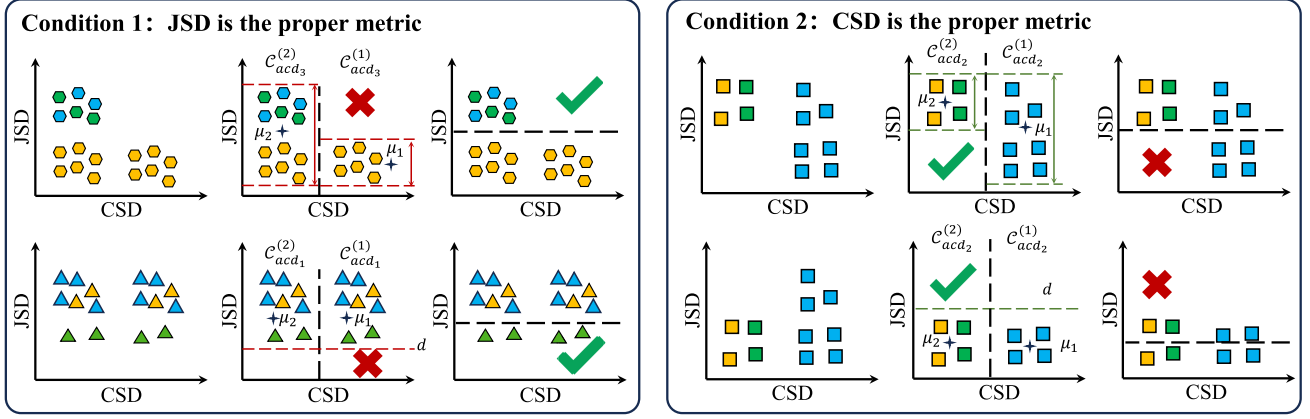


Fig. 3. Conditions of proper metric selection correspond to sample distribution.

Algorithm 1 Adaptive Metric Selection Strategy

Require: Sample set $\mathcal{D}_c$, initial model $\mathcal{F}$

- 1: Obtain $\mathbf{p}_i$ and $\mathbf{f}_i$ for each sample $\mathbf{x}_i$ in $\mathcal{D}_c$.
- 2: Compute JSD and CSD value for each sample $\mathbf{x}_i$ in $\mathcal{D}_c$.
- 3: Obtain $\mathcal{C}_{JSD_c}^1, \mathcal{C}_{JSD_c}^2$ and separation threshold d by applying GMM with values of JSD to all samples in $\mathcal{D}_c$.
- 4: Obtain $\mathcal{C}_{CSD_c}^1, \mathcal{C}_{CSD_c}^2$ by applying GMM with values of CSD to all samples in $\mathcal{D}_c$.
- 5: Calculate the mean μ_1, μ_2 , and standard variance σ_1, σ_2 of JSD for $\mathcal{C}_{CSD_c}^1$ and $\mathcal{C}_{CSD_c}^2$.
- 6: **if** $\sigma_1 < \sigma_2$ and $\mu_1 \leq d \leq \mu_2$ **then**
- 7: $\mathcal{C}_c^1 \leftarrow \mathcal{C}_{JSD_c}^1, \mathcal{C}_c^2 \leftarrow \mathcal{C}_{JSD_c}^2$
- 8: **else if** $\mu_1 \geq d$ and $\mu_2 \geq d$ **then**
- 9: $\mathcal{C}_c^1 \leftarrow \mathcal{C}_{JSD_c}^1, \mathcal{C}_c^2 \leftarrow \mathcal{C}_{JSD_c}^2$
- 10: **else**
- 11: $\mathcal{C}_c^1 \leftarrow \mathcal{C}_{CSD_c}^1, \mathcal{C}_c^2 \leftarrow \mathcal{C}_{CSD_c}^2$
- 12: **end if**
- 13: **return** $\mathcal{C}_c^1, \mathcal{C}_c^2$

JSD within this cluster. In this case, JSD should be used for sample partitioning. In addition, if both $\mu_1 > d$ and $\mu_2 > d$, it suggests that both clusters contain relatively few clean samples, further supporting the use of JSD.

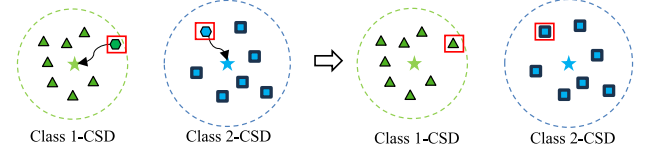
(2) Condition 2: CSD is the proper metric. Conversely, if $\sigma_1 \geq \sigma_2$ and $\mu_1 < d < \mu_2$, this suggests that $\mathcal{C}_{CSD_c}^1$ contains a significant proportion of potential clean samples, and partitioning should be based on CSD. Lastly, if $\mu_1 < d$ and $\mu_2 < d$, JSD-based partitioning may result in the loss of clean samples, making CSD the preferred criterion.

The overall decision process for confident sample selection is summarized in Algorithm 1. After determining the metric, we obtain two sample clusters, $\mathcal{C}_c^1$ and $\mathcal{C}_c^2$. If class c is separated by JSD, $\mathcal{C}_c^2$ is selected as the cleaner cluster, denoted as $\mathcal{D}_c^{\text{conf}}$. Otherwise, $\mathcal{C}_c^1$ is selected as $\mathcal{D}_c^{\text{conf}}$ instead.

C. Label Correction

After obtaining the confident sample set $\mathcal{D}_c^{\text{conf}}$ for each traffic class c , we utilize them to relabel the entire dataset to improve label quality. For the remaining samples $\mathbf{x}$ in

Case 1: Centroid-based correction



Case 2: KNN-based correction

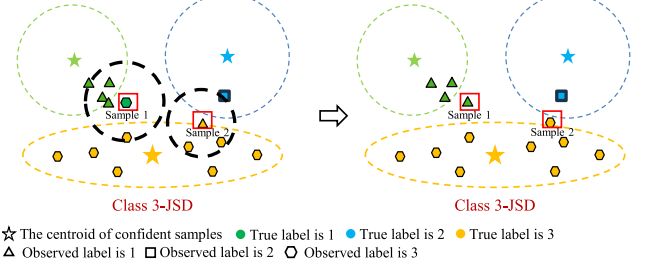


Fig. 4. Motivation for label correction adapted to confident samples and selected metrics for each class.

$\mathcal{D}_{\text{train}}$, we first compute their distances to the confident feature centroids $\mathbf{o}_c$ of each class and identify the closest class c_x .

We show the motivation of our label correction tactic in Figure 4. In case 1 of Figure 4, Class 1 and Class 2 utilize the CSD metric to select confident samples, resulting in compact distributions in latent space. Consequently, the centroids of these samples effectively represent the data distributions of their respective classes. In contrast, Class 3 employs the JSD metric for selecting confident samples, leading to a more dispersed sample distribution and thus, a less reliable centroid. For instance, as shown in case 2 of Figure 4, although Sample 1 is closest to the centroid of Class 3, there are few actual confident samples from Class 3 in its vicinity. Instead, it is surrounded by numerous confident samples from Class 1, prompting us to correct its label to Class 1. On the other hand, Sample 2 is closest to the centroid of Class 3 and is indeed surrounded by a significant number of confident samples from Class 3, validating the use of Class 3 for its label correction.

Based on the above motivation, we present a detailed label correction process as follows. As depicted in case 1 of Figure 4, if the confident samples of class c_x are selected based on metric CSD, it indicates that the feature centroid

Algorithm 2 Parallel Training

Require: Two base neural networks $\mathcal{F}_1, \mathcal{F}_2$ with parameters θ_1, θ_2 ; training data $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, \hat{\mathbf{y}}_i)\}_{i=1}^N$; learning rate η ; batch size B_s ; remember rate function $\lambda(e)$

```

1: for  $e$  in  $(1, \dots, \text{epochs})$  do
2:   Shuffle  $\mathcal{D}_{\text{train}}$  and divide into mini-batches
3:   for  $n$  in  $(1, \dots, t)$  do
4:     Fetch  $n$ -th mini-batch  $\mathcal{B} = \{(\mathbf{x}_i, \hat{\mathbf{y}}_i)\}_{i=1}^{B_s}$ 
5:     Compute per-sample losses:
6:      $\ell_1 = \mathcal{L}_{ce}(\mathcal{B}; \mathcal{F}_1)$ 
7:      $\ell_2 = \mathcal{L}_{ce}(\mathcal{B}; \mathcal{F}_2)$ 
8:     Select small-loss samples:
9:      $\mathcal{B}_1 \leftarrow \text{top } \lfloor \lambda(e)B_s \rfloor$  samples with smallest  $\ell_1$ 
10:     $\mathcal{B}_2 \leftarrow \text{top } \lfloor \lambda(e)B_s \rfloor$  samples with smallest  $\ell_2$ 
11:    Update parameters using opposite model's selection:
12:     $\theta_1 \leftarrow \theta_1 - \eta \nabla_{\theta_1} \mathcal{L}_{ce}(\mathcal{B}_2; \mathcal{F}_1)$ 
13:     $\theta_2 \leftarrow \theta_2 - \eta \nabla_{\theta_2} \mathcal{L}_{ce}(\mathcal{B}_1; \mathcal{F}_2)$ 
14:  end for
15: end for
16: return  $\theta_1, \theta_2$ 

```

of c_x effectively represents the distribution of network flows. Consequently, the centroid is reliable and we directly reassign the label of $\mathbf{x}$ to c_x . Conversely, if the confident samples of class c are selected based on metric JSD, the feature centroid of this class may not be sufficiently reliable. As shown in case 2 of Figure 4, we instead search for the top- k confident samples in the entire confident samples that are closest to $\mathbf{x}$ in feature space. We then assign $\mathbf{x}$ the most frequently occurring class among these top- k samples. If two classes appear with equal frequency, we calculate the average distance from $\mathbf{x}$ to the confident samples of these two classes and assign $\mathbf{x}$ to the class with the smaller average distance.

D. Parallel Training

Using the corrected dataset, we employ parallel training to further mitigate the impact of noisy labels by utilizing a dual-model architecture. This means two distinct base models, which are initialized with the same architecture but different parameters, are trained simultaneously. In the training phase, both models receive the same input samples and generates corresponding predictions. This setup enables the models to learn from the same data from different perspectives. Each network updates its peer using a subset of small-loss samples (i.e., those with the lowest prediction losses), which effectively prioritizes clean samples early in training and mitigates the influence of noisy labels.

The parallel training process is formally described in Algorithm 2. Let the two base models be denoted as $\mathcal{F}_1$ and $\mathcal{F}_2$. During training, both models make predictions on all samples in each mini-batch $\mathcal{B}_s$ and select those with low prediction loss for model updates. Parameter λ is used to control the ratio of samples for model updates, and this parameter is decreased monotonically with epoch in the range of $[0.5, 0.9]$. We utilize the cross-entropy loss $\mathcal{L}_{ce}$ to evaluate

prediction loss which is defined as

$$\mathcal{L}_{ce}(\mathcal{B}; \mathcal{F}) = -\frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{c=1}^C \hat{\mathbf{y}}_{i,c} \log(\mathcal{F}(\mathbf{x}_i; \theta)_c) \quad (5)$$

where $\mathcal{F}(\mathbf{x}_i; \theta)_c$ is the predicted probability of class c from network $\mathcal{F}$ for the input sample $\mathbf{x}_i$, and $\hat{\mathbf{y}}_{i,c}$ is the corresponding observed label $\mathbf{y}_i$ on class c . Finally, we evaluate the identification performance of both models on the corrected data and select the better one as the final model.

VI. EXPERIMENT AND EVALUATION

A. Experimental Setup

1) *Dataset:* We use the following datasets for evaluation:

- **CICIDS-2017** [29] is a widely-used intrusion detection dataset that records the network traffic generated by hundreds of hosts in an internal network and includes the most common attacks such as Web based, Brute force, DoS, DDoS, Infiltration, Bot and Scan.
- **Malicious_TLS** [21] is a encrypted malicious traffic dataset which contains traffic generated by 22 families of malicious code active between 2018 and 2021. In addition, benign TLS traffic is also contained in this dataset. All of this traffic was collected from a real network and was encrypted by TLS.
- **DoHBrw2020** [30] includes normal and malicious DNS-over-HTTPS (DoH) encrypted traffic. The normal traffic is generated by querying benign DNS servers using the DoH protocol, including Cloudflare and Google. The malicious DoH traffic is generated by three different DNS tunneling tools including dns2tcp, DNSCat2, and Iodine.

2) *Noise Settings:* We adopt symmetrical noise settings to create different experimental scenarios, where malicious traffic can be mislabeled as benign while benign traffic can be mislabeled as malicious as well. The quality of labels depends entirely on the capacity of entities and results in different ratios of noise labels. In short, in symmetric noise scenarios, all traffic labels have a certain probability of flipping.

3) *Implementation:* We implement our identification system and all baseline methods using Python 3.8.13. The libraries used include NumPy 1.22.3, PyTorch 1.12.0, scikit-learn 1.1.1, and CUDA 11.8. The models are executed on a Linux server equipped with an Intel® Xeon® Silver 4110 CPU (2.10GHz) and a Tesla V100-PCIE-32GB GPU. We provide a detailed description of the choice of the base model and its parameters in Appendix A. For base model training, the training duration are set at 10 epochs for Malicious_TLS and CICIDS-2017 to avoid overfitting, while the DoHBrw2020 dataset undergoes a longer, 20 epochs training to accommodate its complex features. d is set to 0.5 in confident sample selection. k is set as 10 during label correction. Following the experimental setup in [22], each testing dataset is split in a 7:3 ratio. Labels of certain samples in the training set are intentionally flipped and used as input for BAPTISM during training. Meanwhile, the test set is strictly held out from training and does not contain noisy labels, serving exclusively for model evaluation. To ensure robustness, all experiments are conducted five times with independently randomized training

and testing splits, and the final results are reported as the average over these five runs. During the parallel training phase, the number of training epochs is set to 200.

4) *Comparison Methods*: We compare the performance of BAPTISMs with the following relevant methods.

- **Co-Teaching** [27] utilizes dual neural networks to address low quality training data. Each network selects what it considers the most reliable data samples from each batch and shares them with its peer during training.
- **Co-Learning** [31] uses multiple learners to handle noisy samples. Each model trains on a subset of the data and shares high confidence predictions with others, which helps improve accuracy and resistance to noisy samples by leveraging diverse learner strengths.
- **Co-teaching+** [32] extends the Co-Teaching method by introducing a disagreement-based sample selection mechanism, allowing two networks to collaboratively learn from clean samples while reducing the negative impact of noisy samples, thereby improving model robustness.
- **DivideMix** [33] models the per-sample loss distribution with a Gaussian Mixture Model to distinguish clean and noisy labels, and leverages semi-supervised learning with MixMatch-based label refinement and augmentation, achieving strong robustness against high noise ratios while enhancing model generalization.
- **BoAu** [21] is a boundary-augmentation based approach for malicious traffic identification. It uses all samples involved in decision space construction and decision boundary calculation, enabling the model to maintain a high noise tolerance.
- **MMCo** [18] is a multi-modal co-training approach for malicious traffic detection with noisy labels. It exploits multiple modalities of traffic data and employs parallel, heterogeneous networks to leverage complementary information. By maintaining sufficient disagreement between the networks during training, MMCo mitigates the risk of self-confirmation on noisy samples.
- **CMC** [19] is a noisy malicious traffic detection method based on cross-modal representation consistency. It introduces a consistency constraint between different modal representations to guide confident sample selection. By enforcing agreement across modalities, the method reduces the risk of overfitting to noisy labels and improves the purity of selected samples.
- **MCR** [22] unifies data cleaning and robust training into an ideal representation function approximation. It defines three constraints to let model avoids irrational domain knowledge extraction and ensures strong noisy samples robustness of the representation network.
- **RAPIER** [25] leverage the difference in distribution between benign and malicious traffic data to find the potential clean samples and estimate the possible location of each type of data. To enable a more comprehensive comparison with existing methods, we have extended this approach to support multi-class classification. This extension involves performing density analysis and sample selection separately for each class of malicious traffic. The confident samples obtained from each class

are then aggregated for label correction and model training.

B. Evaluation Metrics

We employ precision (PRE), recall (RE), F1 score (F1), and Macro Accuracy to evaluate the performance of each method.

Suppose the training dataset contains C classes of traffic. Let TP_i , TN_i , FP_i , and FN_i respectively denote the true positives, true negatives, false positives, and false negatives for the i th traffic class ($i \in \{1, \dots, C\}$). For each class i , precision, recall, and F1 score are calculated as

$$\text{PRE}_i = \frac{TP_i}{TP_i + FP_i}, \quad \text{RE}_i = \frac{TP_i}{TP_i + FN_i}$$

$$\text{F1}_i = \frac{2 \times \text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

To mitigate the effects of class imbalance, we employ weighted averages for precision, recall, and F1 score for evaluation by weighting each per-class metric according to the class's support and normalizing by the total sample size.

Macro Accuracy is obtained by averaging the accuracy of each class, where the per-class accuracy is calculated by

$$\text{Accuracy}_i = \frac{TP_i + TN_i}{TP_i + TN_i + FP_i + FN_i}$$

This metric ensures that equal importance is allocated to each class of malicious traffic, facilitating a balanced evaluation of model performance.

C. Comparison Experiment

We systematically present the precision and recall of all comparison methods under varying noise ratio conditions, spanning from 20% to 90%, in Table III. Additionally, we illustrate the F1-score and macro-accuracy respectively in Figures 5 and Figure 6. Here, noise ratio is defined as the proportion of mislabeled samples in all training samples. This comprehensive evaluation yields several key observations:

(1) **BAPTISM exhibits superior performance across all evaluation datasets and noise ratios, demonstrating adaptability to different tasks and experimental settings.** In Table III, BAPTISM achieves the best precision and recall across various datasets and noise ratio settings. Additionally, the method that performs second best varies under different settings, indicating that BAPTISM is robust to various settings. This sustained performance advantage can be attributed to the well-conceived overall design of BAPTISM, where each component contributes to mitigating the impact of noisy samples.

(2) **The model-agnostic property of BAPTISM enhances its scalability and adaptability across various identification scenarios.** The three datasets we employ represent different scenarios: the Malicious_TLS dataset primarily focuses on the communication behaviors of malware; the CICIDS-2017 dataset concentrates on flood attacks; and the DoHBrw dataset represents a more challenging tunnel communication scenario. To accommodate these varied contexts, we adopt different base models for each scenario as mentioned in Appendix A, and achieves the best results across all three datasets. This demonstrates the generalizability of BAPTISM in diverse

TABLE III

THE PRECISION (PRE) AND RECALL (RE) OF ALL METHODS ON THREE EVALUATION DATASETS UNDER VARIOUS NOISE RATIOS (NR)

Dataset	NR	Co-teaching		Co-learning		Co-teaching+		DivideMix		BoAu		MCR		MMCo		CMC		RAPIER		BAPTISM	
		PRE	RE	PRE	RE	PRE	RE	PRE	RE	PRE	RE	PRE	RE	PRE	RE	PRE	RE	PRE	RE	PRE	RE
Malicious_TLS	20%	0.894	0.910	0.934	0.893	<u>0.939</u>	<u>0.939</u>	0.916	0.919	0.920	0.885	0.921	0.913	0.884	0.918	0.897	0.923	0.798	0.811	0.963	0.963
	30%	0.891	0.896	0.863	0.890	<u>0.929</u>	<u>0.925</u>	0.873	0.886	0.907	0.892	0.909	0.907	0.882	0.916	0.887	0.920	0.755	0.806	0.962	0.959
	40%	0.833	0.832	0.872	0.836	<u>0.920</u>	<u>0.922</u>	0.839	0.859	0.917	0.860	0.902	0.899	0.881	0.913	0.878	0.910	0.704	0.688	0.960	0.956
	50%	0.755	0.506	0.750	0.574	0.890	0.899	0.840	0.858	0.893	0.828	<u>0.926</u>	0.891	0.876	<u>0.908</u>	0.894	0.904	0.664	0.586	0.951	0.948
	60%	0.533	0.408	0.559	0.524	0.868	0.882	0.809	0.834	0.882	0.899	<u>0.894</u>	0.882	0.888	0.872	0.884	0.873	0.484	0.462	0.949	0.947
	70%	0.078	0.071	0.426	0.098	0.833	0.858	0.772	0.796	<u>0.882</u>	0.779	0.881	0.853	0.861	0.885	0.873	0.904	0.312	0.358	0.931	0.944
	80%	0.000	0.019	0.086	0.071	0.552	0.612	0.578	0.641	0.845	0.737	<u>0.894</u>	0.859	0.859	0.875	0.871	<u>0.902</u>	0.120	0.147	0.925	0.939
CICIDS-2017	20%	0.939	0.960	0.977	0.971	<u>0.988</u>	<u>0.988</u>	0.981	0.981	0.968	0.938	0.980	0.980	0.967	0.981	0.962	0.976	0.793	0.744	0.993	0.993
	30%	0.920	0.932	0.975	0.974	<u>0.984</u>	<u>0.984</u>	0.974	0.974	0.959	0.911	0.978	0.965	0.954	0.975	0.960	0.975	0.755	0.806	0.989	0.989
	40%	0.968	0.887	0.976	0.936	<u>0.981</u>	<u>0.981</u>	0.964	0.965	0.944	0.925	0.970	0.949	0.952	0.974	0.951	0.973	0.704	0.688	0.987	0.987
	50%	0.971	0.734	0.976	0.862	<u>0.976</u>	<u>0.977</u>	0.960	0.961	0.967	0.919	0.969	0.942	0.947	0.969	0.935	0.954	0.664	0.586	0.982	0.983
	60%	0.643	0.144	0.948	0.234	0.955	<u>0.969</u>	0.942	0.940	0.942	0.935	<u>0.969</u>	0.914	0.923	0.942	0.932	0.950	0.477	0.404	0.978	0.977
	70%	0.084	0.017	0.743	0.067	<u>0.947</u>	<u>0.955</u>	0.931	0.934	0.905	0.858	<u>0.942</u>	0.876	0.869	0.928	0.869	0.927	0.305	0.310	0.960	0.968
	80%	0.000	0.015	0.691	0.030	0.859	0.863	0.835	0.837	0.892	0.841	0.914	0.876	0.865	<u>0.921</u>	0.860	0.918	0.340	0.344	0.956	0.969
DoHBrw2020	90%	0.000	0.014	0.640	0.030	0.156	0.395	0.380	0.108	0.545	0.312	<u>0.911</u>	0.841	0.856	<u>0.912</u>	0.841	0.893	0.387	0.358	0.954	0.967
	20%	0.767	0.761	0.687	0.679	0.762	0.757	0.736	0.723	0.749	0.731	0.786	0.752	0.762	0.749	0.773	<u>0.762</u>	0.643	0.477	0.802	0.794
	30%	0.747	0.741	0.709	0.663	0.761	0.747	0.724	0.712	0.743	0.724	<u>0.777</u>	0.744	0.754	0.741	0.765	<u>0.750</u>	0.518	0.409	0.794	0.788
	40%	0.693	0.573	0.706	0.662	0.696	0.689	0.707	0.698	0.733	0.711	0.738	0.720	0.754	0.741	<u>0.762</u>	<u>0.749</u>	0.488	0.393	0.790	0.784
	50%	0.664	0.389	0.687	0.613	0.677	0.658	0.686	0.678	0.711	0.685	0.727	0.704	0.692	0.668	<u>0.756</u>	<u>0.740</u>	0.360	0.302	0.779	0.776
	60%	0.252	0.248	0.312	0.366	0.623	0.609	0.640	0.625	0.689	0.677	0.716	0.703	0.731	0.713	<u>0.754</u>	<u>0.735</u>	0.342	0.287	0.775	0.766
	70%	0.065	0.255	0.268	0.208	0.344	0.340	0.461	0.439	0.434	0.405	0.671	0.652	0.704	0.676	<u>0.753</u>	<u>0.731</u>	0.326	0.284	0.768	0.761
	80%	0.064	0.253	0.063	0.250	0.293	0.325	0.249	0.244	0.420	0.407	0.645	0.579	0.701	0.672	<u>0.720</u>	<u>0.704</u>	0.322	0.319	0.766	0.757
	90%	0.061	0.248	0.095	0.126	0.133	0.117	0.108	0.109	0.415	0.400	0.624	0.563	0.527	0.502	<u>0.696</u>	<u>0.683</u>	0.294	0.324	0.764	0.757

* The bold text in the table indicates the **best performance**, while the underlined text indicates the **second-best performance**.

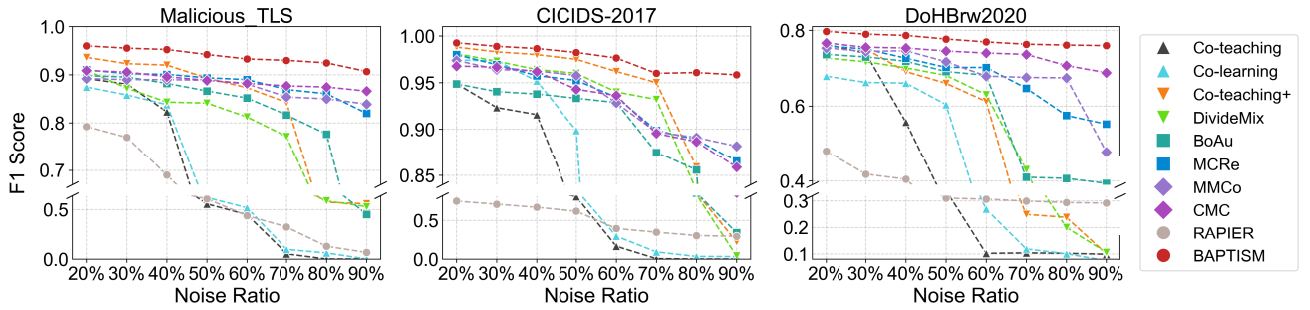


Fig. 5. The F1 Score of all methods under different noise ratios on three evaluation datasets.

identification scenarios. Moreover, as evidenced in Table III and Figure 5, the performance of comparative methods across different identification scenarios is impacted by their dependence on specific model architectures. For instance, BoAu and MCR utilize the MLP-based networks, they perform well on the Malicious_TLS and CICIDS-2017 datasets. However, they falter on the more complex DoHBrw dataset, where the MMCo and CMC surpasses them as they employ more sophisticated CNN and LSTM network to represent traffic.

(3) **The label correction strategy of BAPTISM significantly enhances the quality of training data, thereby improving the model's performance.** Although methods such as Co-teaching, Co-learning, MMCo, and CMC also employ similar parallel training approaches, they struggle to effectively mitigate the impact of noisy samples when the noise ratio exceeds 40%, as shown in Figure 5. Furthermore, the Co-teaching and Co-learning methods even totally lose their identification ability as the noise ratio increases. In contrast, BAPTISM conducts label correction before parallel training, which significantly enhances the quality of the dataset. Consequently, this results in a model with superior identification performance following parallel training across all settings.

(4) **BAPTISM exhibits exceptional performance in identifying each class of malicious traffic by employing adaptive metric selection strategy for confident sample selection.** As depicted in Figure 5, across three datasets, BAPTISM experiences a minimal F1 score degradation of only 0.05 as the noise ratio increases from 20% to 90%, demonstrating greater stability compared to other methods. Moreover, it consistently outperforms in terms of macro accuracy under various noise conditions as shown in Figure 6. Other methods, such as MCR, MMCo, and CMC, rely on a single metric to select confident samples and maintain relatively stable F1 scores; however, their macro accuracy experiences a notable decline as noise ratios increase. Such a limitation highlights their reduced adaptability to class-specific characteristics.

(5) **Compared to RAPIER, BAPTISM effectively utilizes the limited label information available in low-quality data to achieve superior performance.** RAPIER trains an unsupervised model on packet length sequences labeled as benign within the training dataset to explore the differences between benign and malicious traffic in latent space and to identify confident samples. However, this unsupervised approach not only overlooks the residual effective label information in

Malicious_TLS								
Co-teaching	0.84	0.82	0.72	0.55	0.27	0.09	0.04	0.04
Co-learning	0.81	0.80	0.74	0.47	0.28	0.13	0.13	0.04
Co-teaching+	0.89	0.87	0.86	0.83	0.81	0.76	0.47	0.46
DivideMix	0.84	0.79	0.77	0.76	0.72	0.69	0.49	0.47
BoAu	0.89	0.86	0.87	0.84	0.80	0.78	0.71	0.45
MRe	0.88	0.86	0.86	0.85	0.83	0.77	0.78	0.71
MMCo	0.86	0.85	0.84	0.84	0.83	0.80	0.80	0.79
CMC	0.87	0.86	0.84	0.84	0.83	0.83	0.82	0.81
RAPIER	0.71	0.69	0.63	0.58	0.39	0.31	0.18	0.09
BAPTISM	0.94	0.93	0.92	0.91	0.91	0.90	0.89	0.89
	20%	30%	40%	50%	60%	70%	80%	90%
CICIDS-2017								
Co-teaching	0.77	0.76	0.74	0.63	0.19	0.11	0.10	0.10
Co-learning	0.92	0.92	0.85	0.83	0.33	0.13	0.11	0.10
Co-teaching+	0.95	0.94	0.92	0.88	0.85	0.83	0.70	0.10
DivideMix	0.95	0.93	0.91	0.88	0.85	0.82	0.65	0.11
BoAu	0.93	0.83	0.81	0.82	0.79	0.78	0.66	0.28
MRe	0.96	0.93	0.84	0.82	0.82	0.72	0.71	0.66
MMCo	0.89	0.80	0.80	0.79	0.73	0.69	0.63	0.67
CMC	0.82	0.82	0.80	0.76	0.74	0.69	0.63	0.61
RAPIER	0.65	0.60	0.57	0.58	0.38	0.37	0.32	0.29
BAPTISM	0.97	0.96	0.95	0.95	0.94	0.81	0.82	0.81
	20%	30%	40%	50%	60%	70%	80%	90%
DoHBrw2020								
Co-teaching	0.76	0.74	0.57	0.39	0.25	0.25	0.25	0.25
Co-learning	0.68	0.66	0.66	0.61	0.36	0.21	0.25	0.13
Co-teaching+	0.76	0.75	0.69	0.66	0.61	0.34	0.32	0.12
DivideMix	0.72	0.71	0.70	0.68	0.62	0.44	0.25	0.11
BoAu	0.73	0.72	0.71	0.68	0.68	0.41	0.41	0.40
MRe	0.75	0.74	0.72	0.70	0.70	0.70	0.63	0.56
MMCo	0.75	0.74	0.74	0.67	0.71	0.67	0.67	0.50
CMC	0.76	0.75	0.75	0.74	0.73	0.73	0.70	0.68
RAPIER	0.48	0.41	0.39	0.32	0.30	0.30	0.29	0.28
BAPTISM	0.80	0.79	0.78	0.78	0.77	0.76	0.76	0.76
	20%	30%	40%	50%	60%	70%	80%	90%

Fig. 6. The Macro Accuracy of all methods under different noise ratios on three evaluation datasets.

the training data but is also inherently limited to binary classification, necessitating further extension to accommodate multi-class scenarios. Moreover, the unsupervised mechanism struggles to capture complex encrypted communication patterns, as malware often mimics normal encrypted communication behaviors. Consequently, it becomes increasingly ineffective in our experimental setup, particularly under elevated noise conditions, and has difficulty obtaining relatively pure confident samples. As a result, since the label correction module of RAPIER relies on the quality of confident samples, its performance is significantly affected when the quality of these samples is subpar, as shown in Table III and Figure 5.

TABLE IV
THE ABLATION STUDY RESULTS ON MACRO ACCURACY ON THREE EVALUATION DATASETS UNDER VARIOUS NOISE RATIOS (NR)

Datasets	NR	Method				
		w/o PT	w/o LC	w/o JSD	w/o CSD	Full
Malicious_TLS	20%	0.917	0.885	0.923	0.920	0.938
	30%	0.912	0.863	0.903	0.917	0.932
	40%	0.898	0.837	0.908	0.891	0.925
	50%	0.884	0.836	0.903	0.894	0.910
	60%	0.876	0.819	0.873	0.893	0.908
	70%	0.871	0.811	0.851	0.851	0.905
	80%	0.851	0.796	0.840	0.858	0.894
	90%	0.789	0.753	0.835	0.835	0.892
CICIDS-2017	20%	0.892	0.822	0.958	0.956	0.974
	30%	0.908	0.795	0.943	0.936	0.962
	40%	0.872	0.821	0.928	0.922	0.952
	50%	0.821	0.797	0.915	0.918	0.949
	60%	0.791	0.796	0.896	0.896	0.938
	70%	0.751	0.739	0.797	0.797	0.854
	80%	0.747	0.722	0.790	0.788	0.816
	90%	0.743	0.680	0.777	0.789	0.805
DoHBrw2020	20%	0.770	0.757	0.779	0.773	0.795
	30%	0.756	0.751	0.756	0.758	0.787
	40%	0.745	0.734	0.757	0.758	0.783
	50%	0.699	0.715	0.746	0.753	0.775
	60%	0.691	0.701	0.741	0.738	0.765
	70%	0.655	0.676	0.734	0.728	0.760
	80%	0.634	0.667	0.737	0.734	0.757
	90%	0.508	0.646	0.736	0.716	0.757

D. Ablation Study

To demonstrate the effectiveness of each component in BAPTISM, we perform three separate ablation analysis experiments. The comparison models are used as follows: (1) Full component of BAPTISM (Full); (2) without parallel training (w/o PT); (3) without label correction (w/o LC); (4) without metric selection and only use CSD to filter clean samples (w/o JSD); (5) without metric selection and only use JSD to filter clean samples (w/o CSD). Figure 7 shows the F1 score of ablation study on three evaluation datasets under each noise ratio settings, while Table IV shows the corresponding macro accuracy results. We can draw the following conclusions:

(1) BAPTISM with the full components outperforms other comparison models under all experimental settings, showing the soundness of our proposed framework. When we remove any other component of BAPTISM, the F1 decreases range from 0.05 to 0.25 and the macro accuracy decreases range from 0.014 to 0.058. This indicates the synergy of adaptive metric selection, label correction, and parallel training, which collectively enhance the model's ability to accurately identify malicious traffic despite the presence of noisy samples.

(2) From the results of BAPTISM without Parallel Training, we observe that this training strategy further mitigate the impact of noise samples. When the parallel training component is removed, the F1 score experiences a decrease ranging from 0.1 to 0.5, while macro accuracy decreases from 0.02 to 0.25. This performance degradation can be attributed to the effectiveness of using low-loss samples for model updates, as the corrected dataset still contains a certain number of noisy samples which will be discussed in section VI-E.

(3) Excluding Label Correction component results in a performance degradation, highlighting a performance bottleneck when relying solely on the parallel training to address noisy samples. This underscores the importance of the volume of confident samples. Although utilizing parallel training can

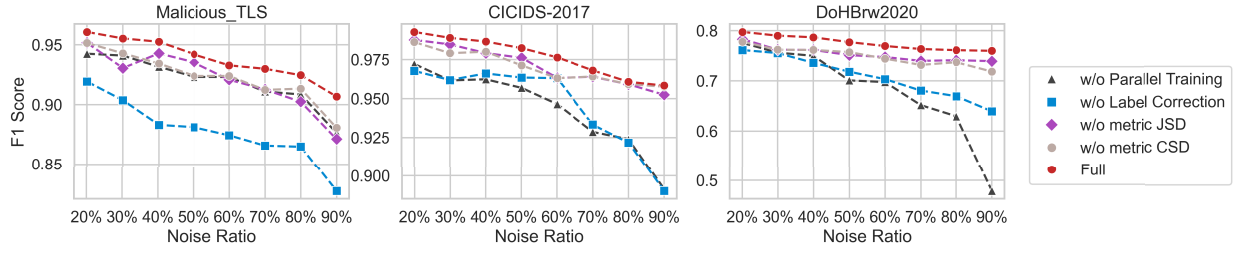


Fig. 7. The F1 scores of all methods for ablation study under different noise ratios on three evaluation datasets.

partially mitigate the impact of low-quality data, there remains a macro accuracy gap ranging from 0.037 to 0.152 compared to the full model, as shown in Table IV. Moreover, this performance gap widens as the noise ratio increases, further emphasizing the importance of this component.

(4) The adaptive metric selection strategy is crucial for enhancing the model's robustness. As shown in Table IV, whether using JSD alone or CSD alone, the model's macro accuracy decreases, as each metric has varying effectiveness in selecting confident samples for different classes of malicious traffic. This demonstrates the need for metric selection, which allows for more suitable metric to be chosen for filtering confident samples, thereby improving the overall performance of the model. Furthermore, the results in Figure 7 illustrates that the model performance fluctuates and lacks stability.

E. Performance on Confident Sample Selection

To further analyze the performance advantages of BAPTISM, we also evaluate its effectiveness in confident sample selection and label correction. Among the comparison methods considered, MMCo, CMC, and RAPIER incorporate a confident sample selection mechanism. Specifically, MMCo and CMC utilize these selected samples to update the model, whereas RAPIER and our proposed method leverage confident samples to refine the labels of the entire dataset.

We first demonstrate the advantages of BAPTISM in confident sample selection. As shown in Figure 8, BAPTISM consistently outperforms all compared methods by selecting a greater number of clean samples while containing fewer noisy samples. This results in a significant advantage in both the quantity and purity of the selected confident samples. Specifically, when the noise ratio ranges from 20% to 70%, over 90% of the samples selected by BAPTISM are truly clean. Even when the noise ratio exceeds 70%, although the number of noisy samples within the selected confident set increases, the number of clean samples remains substantially higher than that of all comparison methods.

In contrast, the RAPIER method achieves moderate confident sample purity when the noise ratio is below 40%, but it selects fewer clean samples compared to BAPTISM. Moreover, across different datasets and noise settings, RAPIER consistently includes a considerable number of noisy samples in its selected samples, and this issue becomes more severe as the noise ratio increases. Meanwhile, MMCo and CMC achieve relatively high overall purity in confident sample selection. Compared to RAPIER, these two methods include fewer

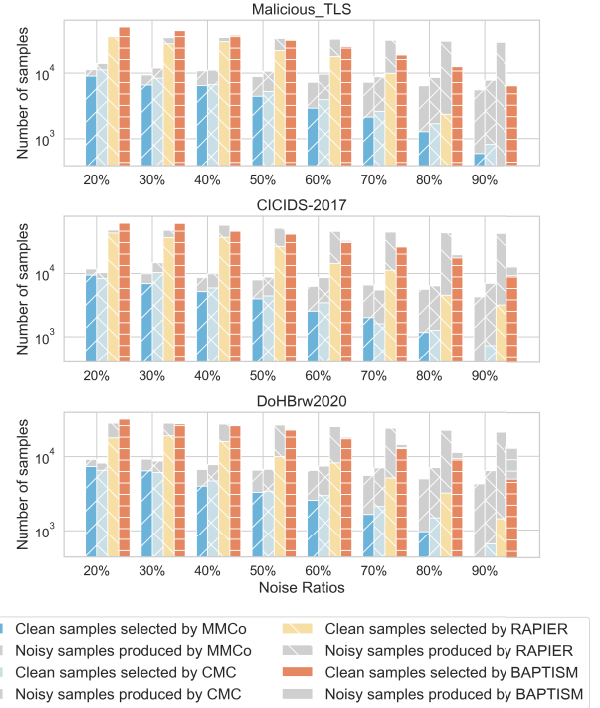


Fig. 8. The purity of confident samples obtained by all comparison methods under different noise ratio settings.

noisy samples in their selected confident samples, thereby reducing their negative impact on model training. However, they select far fewer confident samples than BAPTISM, leading to insufficient training and suboptimal performance. This performance gap is particularly pronounced in macro accuracy, as certain malicious traffic classes inherently have fewer samples, and excessive filtering further reduces their representation.

Furthermore, as illustrated in Figure 8, the sample purity of both MMCo and CMC decreases significantly when the noise ratio exceeds 70%. This demonstrates that in high noise ratio settings, the performance of sample selection strategies based on low loss is limited, which substantially impacts the model update stage due to the influence of noisy labels, thereby degrading overall model performance. Conversely, BAPTISM utilizes an adaptive metric selection strategy that still manages to obtain a sufficient number of confident samples with higher purity even in scenarios with high noise ratios. This capability underpins the superior performance of our framework. Regarding the RAPIER method, it employs a density-based sample selection strategy, which is particularly suitable for flood attack

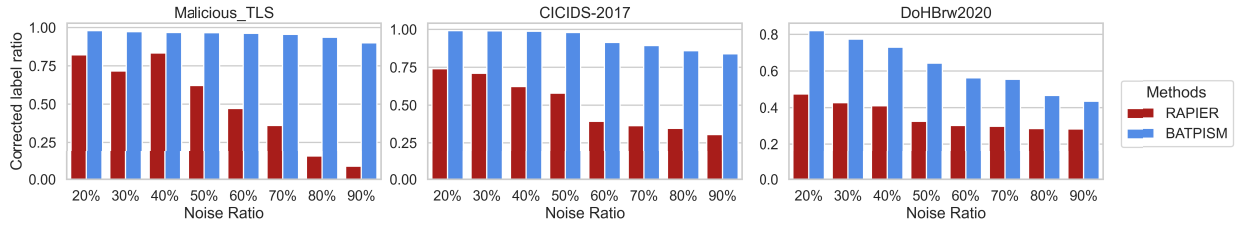


Fig. 9. The correct label ratios after label correction under different noise ratio settings.

scenarios. As evidenced in the CICIDS-2017 dataset, it performs best in sample selection, aligning with its characteristics. However, RAPIER’s performance deteriorates when dealing with more complex encrypted communication behavior data. This decline is notably apparent in the Malicious_TLS dataset, where it almost fails at a noise ratio of 90%.

F. Performance on Label Correction

We further evaluate the model’s performance in label correction to demonstrate each model’s capability to refine low-quality training data. We conduct a comparative evaluation of BAPTISM and RAPIER, as other comparison methods do not include this process. Given a low-quality training dataset, we apply each method to correct the labels and compute the proportion of correctly labeled samples in the refined dataset.

The label noise correction results are shown in Figure 9. BAPTISM significantly reduces the original noise ratio to lower levels. Across all noise settings, BAPTISM consistently increases the ratio of correct labels to over 80% on Malicious_TLS and CICIDS-2017 datasets. On DoHBrw2020 dataset, BAPTISM improves the ratio of correct labels by 2% to 34%. These experimental results demonstrate the robust adaptability of our proposed labeling strategy across diverse datasets and varying noise settings. In contrast, RAPIER exhibit a significant decline in label correction performance as the noise ratio increase. On Malicious_TLS dataset, RAPIER almost lose their ability to correct training dataset.

The factors leading to performance differences between these methods can be attributed to two main aspects: On the one hand, both BAPTISM and RAPIER rely on the quality of confident samples. As observed from the results in the previous chapter, when the noise ratio exceeds 60%, the confident samples selected by RAPIER contain a significant amount of noisy samples as shown in Figure 8, which greatly impacts their purity. This is the fundamental reason for the notable performance gap between RAPIER and BAPTISM after subsequent label correction phases. On the other hand, regarding label correction strategies, RAPIER trains a series of ML models on the selected confident samples and employs a voting mechanism for data label correction. The effectiveness of this step depends on the purity of the confident samples obtained in the previous step. Since the ML models they used do not specifically address noisy samples, their decision-making results are biased against the actual data distribution, leading directly to poor outcomes in label correction.

We further elucidate the limitations of using machine learning methods to correct labels. As depicted in Figure 8, RAPIER selects confident samples with relatively high purity

when the noise ratio is below 60%. However, as shown across various datasets in Figure 9, there is a noticeable discrepancy between the purity of the confident samples and their purity after label correction. This indicates that even a lower proportion of noisy labels can be detrimental to the performance of machine learning algorithms. With higher noise ratios, the accuracy of RAPIER’s label correction sharply declines, nearly rendering the label correction capability ineffective. Moreover, RAPIER almost fails in the case of the Malicious_TLS dataset when the noise ratio exceeds 80%, due to the challenging nature of extracting encrypted communication characteristics. While RAPIER’s design primarily relies on analyzing network flow density to detect malicious activity, which is suitable for flood attacks, it lacks sufficient capability to mine communication characteristics, resulting in sub-optimal performance.

VII. HYPER-PARAMETER SENSITIVITY ANALYSIS

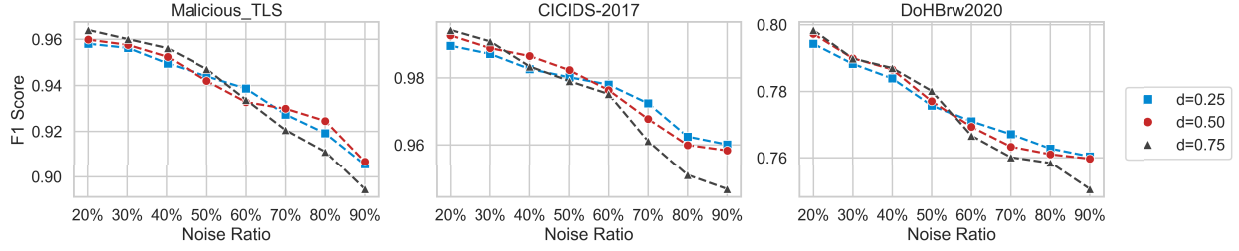
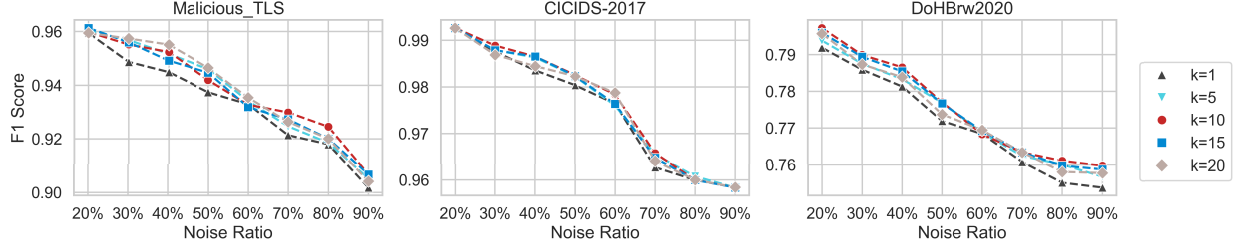
A. Sensitivity to the Threshold d in Confident Sample Selection

We investigated the sensitivity of our framework to the threshold parameter d used in confident sample selection. Intuitively, a larger value of d corresponds to selecting more samples as confident, while a smaller value enforces a stricter criterion and results in fewer selected samples. To analyze its effect, we evaluated three representative settings, $d = 0.25, 0.5, 0.75$, across multiple datasets and noise ratios, with adopted as the default configuration in our framework.

The results are summarized in Fig. 10. Across the three settings of d , the overall performance differences remain small, with the maximum variation in F1 not exceeding 0.15. This observation shows that the framework is reasonably robust to the choice of d within the range of $[0.25, 0.75]$. A closer inspection reveals several trends. When the noise ratio is below 50%, setting $d = 0.75$ sometimes achieves higher performance, as a larger threshold allows more clean samples to be utilized for training. However, as the noise ratio increases, the performance of $d = 0.75$ degrades more rapidly, reflecting the higher risk of including noisy samples. On the other hand, $d = 0.25$ enforces stricter purity by reducing the number of confident samples, but this conservative strategy often leads to suboptimal results, highlighting that the quantity of confident samples is also crucial for robust training.

B. Sensitivity to the Number of Neighbors k in Label Correction

we further evaluated k in $[1, 5, 10, 15, 20]$ on all evaluation datasets. As summarized in Fig 11, the overall performance

Fig. 10. The F1 score of different d in confident sample selection.Fig. 11. The F1 score of different k in label correction.

of our method remains stable across the range $k = 5 \sim 20$, suggesting that the framework is not highly sensitive to the exact value of k .

The only consistent degradation appears when $k = 1$. In this case, the model relies on a single neighbor as the reference, which makes the estimation unstable and more vulnerable to noisy samples. By contrast, when more neighbors are considered (e.g., $k \geq 5$), the reference distribution becomes smoother and more representative, leading to reliable label correction. Beyond $k = 10$, no persistent significant improvements were observed, indicating that moderately small values of k are sufficient in practice. This analysis demonstrates that our approach is robust to a wide range of k , and practitioners can select k within a reasonable interval (e.g., $5 \sim 20$) without significant performance variation.

VIII. THE COMPLEXITY OF BAPTISM

In this section, we analyze the complexity of BAPTISM. Let N be the total number of training samples, C the number of classes, D the feature dimension of each sample, $|\theta_{\mathcal{F}}|$ the parameter count of the base model $\mathcal{F}$, n_c the number of confident samples in class c , $M_{\text{conf}} = \sum_{c=1}^C n_c$ the total number of confident samples, R the number of samples to be relabeled (worst-case $R = N$), $\alpha \in [0, 1]$ the fraction of samples that trigger the fallback, E the number of training epochs, and B_s the mini-batch size. The computational cost of BAPTISM mainly arises from three components.

A. Confident Sample Selection

First, computing the feature vectors $\mathbf{f}_i$ and predicted probabilities $\mathbf{p}_i$ for all N samples via $\mathcal{F}$ requires $O(N|\theta_{\mathcal{F}}|)$. Next, evaluating the JSD against one-hot labels over C classes costs $O(NC)$. Constructing class centroids and computing the CSD over D -dimensional features costs $O(ND)$. For each class, fitting a 2-component Gaussian Mixture Model (GMM) on the

JSD and CSD scores using T_{EM} EM iterations costs $O(T_{\text{EM}}N)$ in total. Finally, computing means and variances of JSD within the two CSD clusters and applying the selection rules is linear in the class size, $O(N)$. Summing all contributions, the overall time complexity of confident sample selection is $O(N(|\theta_{\mathcal{F}}| + \max\{C, D\} + T_{\text{EM}}))$. The peak memory is dominated by storing features ($N \times D$), predicted probabilities ($N \times C$), and class centroids ($C \times D$): $S_{\text{peak}} = O(ND + NC + CD)$.

B. Label Correction

Given the features and centroids already computed, assigning each sample to its nearest class centroid costs $O(RCD)$. For the αR samples whose predicted class relies on the fallback mechanism, performing top- k voting over the confident sample pool via naive search costs $O(\alpha R M_{\text{conf}} D)$. Therefore, the overall time complexity is $O(RD(C + \alpha M_{\text{conf}}))$, which in the worst case becomes $O(N^2 D)$. The peak memory is dominated by storing the feature matrix ($O(ND)$), centroids ($O(CD)$), and the confident sample pool ($O(M_{\text{conf}} D)$).

C. Parallel Training

The time complexity of Parallel Training is dominated by the forward and backward passes of both base models, giving $O(E \cdot N \cdot |\theta_{\mathcal{F}}|)$, where E is the number of epochs. Operations such as mini-batch shuffling and top- k sample selection are negligible compared to gradient computations. The peak memory usage is $O(|\theta_{\mathcal{F}}| + B_s \cdot D)$, accounting for the model parameters and a single mini-batch of feature vectors.

In summary, the total time complexity of BAPTISM can be approximated as $O(N(|\theta_{\mathcal{F}}| + \max\{C, D\} + T_{\text{EM}}) + RD(C + \alpha M_{\text{conf}}) + EN|\theta_{\mathcal{F}}|)$, with peak memory usage of $O(ND + NC + CD + M_{\text{conf}} D + B_s D + |\theta_{\mathcal{F}}|)$. In practice, the dominant runtime cost comes from the forward and backward passes during Parallel Training, which scales linearly with the number of training samples and model parameters.

TABLE V
THE PARAMETERS OF THE BASE MODEL FOR DIFFERENT IDENTIFICATION TASK

Base Model	Layer Type	Parameters	Activation	Normalization	Pooling/Dropout
MLP (Malicious_TLS and CICIDS-2017)	Linear	input_dim→128	ReLU	-	-
	Linear	128→64	ReLU	-	-
	Linear	64→32	ReLU	-	-
	Linear	32→num_class	None	-	-
CNN (DoHBrw2020)	Conv1d	1→128, kernel=3, stride=1, pad=1	LeakyReLU (0.01)	BatchNorm1d(128)	-
	Conv1d	128→128, kernel=3, stride=1, pad=1	LeakyReLU (0.01)	BatchNorm1d(128)	MaxPool1d(2,2) + Dropout(0.25)
	Conv1d	128→256, kernel=3, stride=1, pad=1	LeakyReLU (0.01)	BatchNorm1d(256)	MaxPool1d(2,2) + Dropout(0.25)
	Conv1d	256→512, kernel=3, stride=1, pad=0	LeakyReLU (0.01)	BatchNorm1d(512)	-
	Conv1d	512→128, kernel=3→2, stride=1	LeakyReLU (0.01)	BatchNorm1d(256→128)	AvgPool1d(adaptive)
	Linear	128→num_class	-	-	-

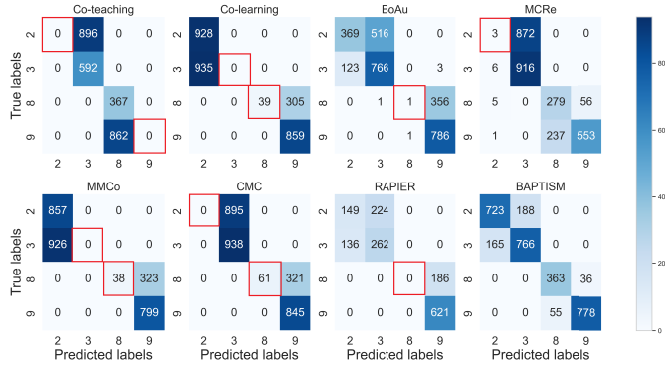


Fig. 12. Confusion matrix of BAPTISM on the Malicious_TLS dataset with a noise ratio of 40%. In this figure, the numbers 2 and 3 correspond to Caphaw.AH and Caphaw.A, while 8 and 9 represent Panda.BZA!tr and PandaZeusCC. We have marked the severe cases of misclassification with red boxes.

IX. DISCUSSION

In this section, we further discuss the limitations of our method. First, we conduct a more detailed analysis of the identification performance of each method. We find that all methods trained on low-quality training datasets struggle to accurately identify malicious traffic classes that belong to the same family. We illustrate this issue by examining the identification performance of each method on the Malicious_TLS dataset, which comprises 22 classes of malicious traffic, under the noise ratio of 40%. Among these, Caphaw.AH and Caphaw.A belong to the Caphaw family, while Panda.BZA!tr and PandaZeusCC are part of the Panda family. Figure 12 displays the confusion matrices for these four classes of malicious traffic for each compared method. Additionally, we present the performance of BAPTISM across all classes in Appendix B to better highlight the difficulty of distinguishing between malicious traffic from the same family.

As demonstrated in Figure 12, although BAPTISM generates a relatively high number of false positives between these classes, it remains the optimal choice compared to other methods. This is because the alternative approaches consistently misclassify entire class as other classes. For instance, methods such as BoAu and RAPIER predominantly misclassify both Panda.BZA!tr and PandaZeusCC, erroneously categorizing them as other classes. Similarly, MCRc tends to completely misclassify the classes Caphaw.AH and Caphaw.A. Other comparative methods fail to differentiate accurately

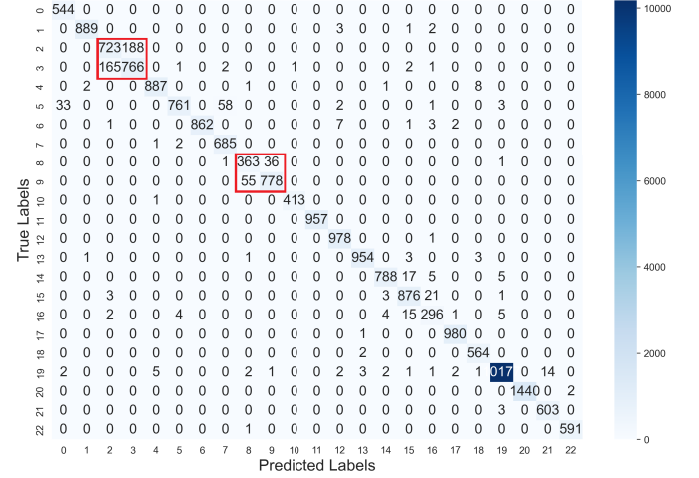


Fig. 13. Full confusion matrix of BAPTISM on Malicious_TLS Dataset with noise ratio of 0.4, where the numbers 2 and 3 correspond to Caphaw.AH and Caphaw.A, while 8 and 9 represent Panda.BZA!tr and PandaZeusCC.

between these scenarios, demonstrating a common inability to distinguish these specific classes.

The inability of existing methods to accurately identify these traffic classes stems from the inherent code reuse among malware variants within the same family, which results in similar traffic patterns and complicates differentiation, particularly when noisy samples are present. This phenomenon underscores the limitations of current methods in effectively identifying such closely related variants. To address this issue, one potential approach is to initially group these similar variants into broader families, facilitating more robust classification and analysis. Alternatively, incorporating additional expert knowledge could enhance the model's ability to differentiate between these closely related classes. Further research is warranted to develop techniques that can effectively mitigate the impact of code reuse and improve the discrimination of malware variants within the same family.

Second, it is worth noting that there exist cases where neither the confidence-based metric (JSD) nor the feature-based metric (CSD) can clearly separate the samples. This phenomenon is particularly evident in the DoHBrw2020 dataset. Even with a relatively low noise ratio of 20%, the model's performance remains lower compared to the other two datasets. The main reason lies in the intrinsic characteristics of DoH traffic: it is inherently obfuscated and stealthily encapsulated, which increases the difficulty of distinguishing

certain classes. Consequently, both JSD and CSD metrics may fail to provide a clear separation, leading to potential misclassifications. This observation highlights that the fallback behavior of the framework is not a limitation of our approach but rather reflects the inherent challenges posed by certain types of network traffic.

X. CONCLUSION

In this paper, we propose a robust framework, BAPTISM, to identify encrypted malicious traffic using low-quality training data. This framework is independent of model architecture and adaptable to various settings. Specifically, we implement early representation for training data by training an appropriate base model with early stopping to generate traffic representation. Instead of relying on single metric, we employ bi-metric, JSD and CSD, to analyze traffic representation characteristics from distinct perspectives. We then design an adaptive metric selection strategy to identify the more proper metric for each class, applying it to select confident samples. Based on the confident samples and selected metric for each class of traffic, we perform label correction adapt to class nature to improve the quality of training data. Additionally, we implement a parallel training strategy to train the base model with the corrected data, thus mitigating the impact of remaining noisy samples. Experimental results demonstrate that BAPTISM can adapt to different base models and surpass others under various noise ratio settings. Furthermore, it consistently selects confident samples with the highest purity and volume across different experimental settings.

APPENDIX A PARAMETERS OF BASE MODELS

Referring to the previous research [18], [19], [20], [21], [22], we employ a MLP as the base model for identification tasks on the Malicious_TLS and CICIDS-2017 datasets. For DoHBrw2020 dataset, we utilize a CNN as the base model. We provide the detailed list of parameters for base models used in BAPTISM in Table V, where ‘input_dim’ denotes the dimensionality of the original features in the dataset, while ‘num_class’ indicates the number of classes included in the dataset.

APPENDIX B FULL RESULT OF CONFUSION MATRIX OF BAPTISM ON MALICIOUS_TLS DATASET

We present a full confusion matrix from BAPTISM on the Malicious_TLS dataset under noise ratio of 0.4 to illustrate the sources of false positives, as shown in the Figure 13. As mentioned in the Discussion section, the false positives generated by BAPTISM primarily arise from traffic classes which belongs to the same family. In contrast, false positives are significantly less frequent for other classes of malicious traffic. This indicates that BAPTISM can effectively capture the distinct patterns between different classes of malicious traffic under low-quality training data. However, for classes belonging to the same family, additional information is needed for effective identification.

REFERENCES

- [1] (Jan. 2024). *Malicious Traffic Detection: A Guide for Businesses*. Accessed: Mar. 8, 2025. [Online]. Available: <https://no7-svip.urlapi-dodo.me/s?t=b2b8fab84d31c1111e759d6a8c76675f>
- [2] Zscaler. *Threatlabz-Encrypted-Attacks-Report*. Accessed: Jan. 5, 2026. [Online]. Available: <https://www.zscaler.com/campaign/threatlabz-encrypted-attacks-report>
- [3] R. T. El-Maghraby, N. M. A. Elazim, and A. M. Bahaa-Eldin, “A survey on deep packet inspection,” in *Proc. 12th Int. Conf. Comput. Eng. Syst. (ICCES)*, Dec. 2017, pp. 188–197.
- [4] P. Prasse, L. Machlica, T. Pevný, J. Havelka, and T. Scheffer, “Malware detection by analysing encrypted network traffic with neural networks,” in *Proc. Mach. Learn. Knowl. Discovery Databases, Eur. Conf.*, Skopje, Macedonia, Sep. 2017, pp. 73–88.
- [5] B. Anderson and D. McGrew, “Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity,” in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2017, pp. 1723–1732.
- [6] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, “Malware traffic classification using convolutional neural network for representation learning,” in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.
- [7] Z. Fu et al., “Encrypted malware traffic detection via graph-based network analysis,” in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2022, pp. 495–509.
- [8] Z. Wang and V. L. L. Thing, “Feature mining for encrypted malicious traffic detection with deep learning and other machine learning algorithms,” *Comput. Secur.*, vol. 128, May 2023, Art. no. 103143.
- [9] C. Rong, G. Gou, M. Cui, G. Xiong, Z. Li, and L. Guo, “MalFinder: An ensemble learning-based framework for malicious traffic detection,” in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2020, p. 7.
- [10] C. Fu, Q. Li, M. Shen, and K. Xu, “Realtime robust malicious traffic detection via frequency domain analysis,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.
- [11] C. Fu, Q. Li, M. Shen, and K. Xu, “Frequency domain feature based robust malicious traffic detection,” *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.
- [12] W. Wang et al., “HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection,” *IEEE Access*, vol. 6, pp. 1792–1806, 2018.
- [13] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, “CBSeq: A channel-level behavior sequence for encrypted malware traffic detection,” *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.
- [14] S. Cai, H. Xu, M. Liu, Z. Chen, and G. Zhang, “A malicious network traffic detection model based on bidirectional temporal convolutional network with multi-head self-attention mechanism,” *Comput. Secur.*, vol. 136, Jan. 2024, Art. no. 103580.
- [15] Y. Hong, Q. Li, Y. Yang, and M. Shen, “Graph based encrypted malicious traffic detection with hybrid analysis of multi-view features,” *Inf. Sci.*, vol. 644, Oct. 2023, Art. no. 119229.
- [16] B. Wang, J. Zhang, Z. Zhang, W. Luo, and D. Xia, “Robust traffic classification with mislabelled training samples,” in *Proc. IEEE 21st Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2015, pp. 328–335.
- [17] B. Wang, J. Zhang, Z. Zhang, L. Pan, Y. Xiang, and D. Xia, “Noise-resistant statistical traffic classification,” *IEEE Trans. Big Data*, vol. 5, no. 4, pp. 454–466, Dec. 2019.
- [18] Q. Yuan, G. Gou, Y. Zhu, and Y. Wang, “MMCo: Using multimodal deep learning to detect malicious traffic with noisy labels,” *Frontiers Comput. Sci.*, vol. 18, no. 1, Feb. 2024, Art. no. 181809.
- [19] Q. Yuan, W. Niu, Y. Wang, G. Gou, and B. Lu, “Malicious traffic detection with noise labels based on cross-modal consistency,” *IEEE Netw. Lett.*, vol. 6, no. 2, pp. 148–151, Jun. 2024.
- [20] Q. Yuan et al., “ULDC: Unsupervised learning-based data cleaning for malicious traffic with high noise,” *Comput. J.*, vol. 67, no. 3, pp. 976–987, Apr. 2024.
- [21] Q. Yuan et al., “BoAu: Malicious traffic detection with noise labels based on boundary augmentation,” *Comput. Secur.*, vol. 131, Aug. 2023, Art. no. 103300.
- [22] Q. Yuan, G. Gou, Y. Zhu, Y. Zhu, G. Xiong, and Y. Wang, “MCRc: A unified framework for handling malicious traffic with noise labels based on multidimensional constraint representation,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 133–147, 2024.
- [23] L. Yao, A. Hou, W. Niu, Q. Yuan, J. He, and Y. Zhang, “Gedss: A generic framework to enhance model robustness for intrusion detection on noisy data,” in *Proc. 27th Int. Conf. Comput. Supported Cooperat. Work Design (CSCWD)*, May 2024, pp. 1645–1650.

- [24] L. Yao, W. Niu, Q. Yuan, B. Li, Y. Zhang, and X. Zhang, "A robust malicious traffic detection framework with low-quality labeled data," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2024, pp. 2719–2724.
- [25] Y. Qing et al., "Low-quality training data only? A robust framework for detecting encrypted malicious network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Feb. 2024.
- [26] G. Han, H. Zhang, Z. Zhang, Y. Ma, and T. Yang, "AI-based malicious encrypted traffic detection in 5G data collection and secure sharing," *Electronics*, vol. 14, no. 1, p. 51, Dec. 2024. [Online]. Available: <https://www.mdpi.com/2079-9292/14/1/51>
- [27] B. Han et al., "Co-teaching: Robust training of deep neural networks with extremely noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8527–8537.
- [28] Y. Lu, Y. Zhang, B. Han, Y.-M. Cheung, and H. Wang, "Label-noise learning with intrinsically long-tailed data," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 1369–1378.
- [29] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, pp. 108–116, May 2018.
- [30] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. H. Lashkari, "Detection of DoH tunnels using time-series classification of encrypted traffic," in *Proc. IEEE Int. Conf. Depend. Auton. Secure Comput. Int. Conf. Pervasive Intell. Comput. Int. Conf. Cloud Big Data Comput. Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCCom/CyberSciTech)*, Jun. 2020, pp. 63–70.
- [31] C. Tan, J. Xia, L. Wu, and S. Z. Li, "Co-learning: Learning from noisy labels with self-supervision," in *Proc. 29th ACM Int. Conf. Multimedia*, Oct. 2021, pp. 1405–1413.
- [32] X. Yu, B. Han, J. Yao, G. Niu, I. W. Tsang, and M. Sugiyama, "How does disagreement help generalization against label corruption?," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7164–7173.
- [33] J. Li, S. C. H. Hoi, and R. Socher, "DivideMix: Learning with noisy labels as semi-supervised learning," in *Proc. 8th Int. Conf. Learn. Represent.*, Addis Ababa, Ethiopia, Apr. 2020. [Online]. Available: <https://openreview.net/forum?id=HJgExaVtwr>



Xiang Luo received the B.S. degree from the University of Science and Technology Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include network traffic analysis and intrusion detection.



Chang Liu received the Ph.D. degree from the University of Chinese Academy of Sciences in 2020. She is currently a Senior Engineer with the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include network traffic analysis and information security.



Gang Xiong is currently a Full Professor and the Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences. He has authored more than 110 papers in refereed journals and conference proceedings. His research interests include cyber security, network measurement, network traffic analysis, and network forensics.



Gaopeng Gou received the B.Eng., M.Eng., and Ph.D. degrees from Beihang University, China, in 2005, 2008, and 2014, respectively. He is currently a Full Professor and the Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network security and anomaly detection.



Zhen Li received the B.Eng. degree from Shandong University, China, in 2009, the M.S. degree from the Institute of Computing Technology, Chinese Academy of Sciences, China, in 2012, and the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, in 2020. His research interests include encrypted network behavior analysis and network measurement.



Junzheng Shi received the M.Eng. degree from Beijing University of Posts and Telecommunications, China, in 2013, and the Ph.D. degree from the University of Chinese Academy of Sciences in 2021. He is currently a Senior Researcher with the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include network measurement and behavior analysis.



Li Guo is currently a Full Professor and the Ph.D. Supervisor with the Institute of Information Engineering, Chinese Academy of Sciences, China. She is also the Director of the Institute of Information Engineering, Chinese Academy of Sciences. Her research interests include data stream management systems and data mining.



Binxing Fang received the M.Eng. degree from Tsinghua University, China, in 1984, and the Ph.D. degree from Harbin Institute of Technology, China, in 1989. He is currently a member of Chinese Academy of Engineering and the Chief Scientist of China Electronics Corporation. His research interests include network attack and defense, cyber security, and cloud computing security.